

TUẦN 3: MVC Architecture - Tài Liệu Lý Thuyết Chi Tiết

Giáo viên hướng dẫn: Senior Java Developer Cấp độ: Nâng cao (Tiếp theo Tuần 1-2) Thời lượng: 1 tuần (7 phiên)

Nội dung Tuần 3

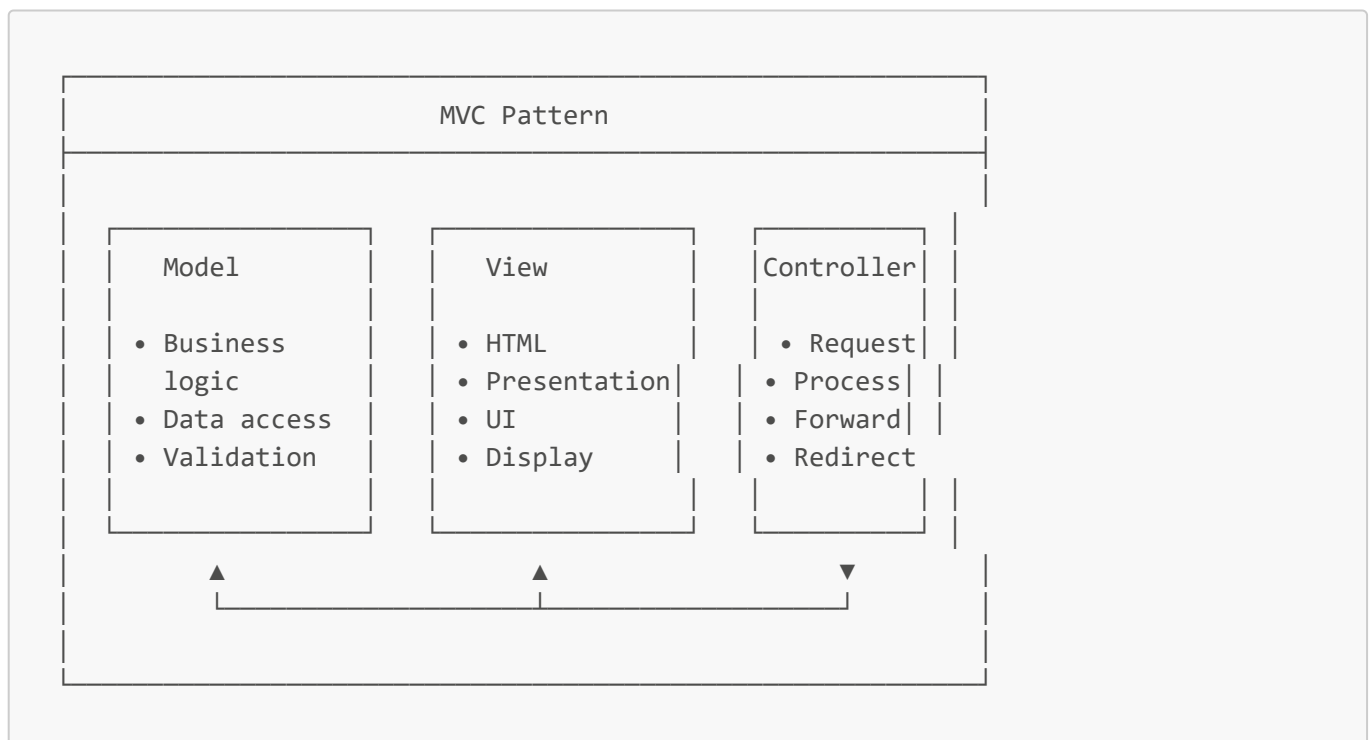
- **Session 15:** MVC Pattern (Mô hình)
 - **Session 16:** Forward vs Redirect
 - **Session 17:** JavaBeans
 - **Session 18:** Controller Organization
 - **Session 19:** Validation (Kiểm thử dữ liệu)
 - **Session 20:** Error Handling (Xử lý lỗi)
 - **Session 21:** Mini Project
-

SESSION 15: MVC Pattern

15.1 MVC là gì?

MVC = Model View Controller

Đây là một **architectural pattern** (kiến trúc thiết kế) để tổ chức code, tách biệt logic từ giao diện.



15.2 Ba thành phần MVC

1. Model (Mô hình - Dữ liệu)

Mục đích: Lưu trữ dữ liệu và logic xử lý.

Bao gồm:

- JavaBeans (POJO - Plain Old Java Objects)
- Business Logic
- Data Access Objects (DAO)
- Database

Ví dụ:

```
public class User {  
    private int id;  
    private String username;  
    private String email;  
  
    // Getters/Setters  
    public String getUsername() { return username; }  
    public void setUsername(String u) { this.username = u; }  
}
```

2. View (Giao diện - Hiển thị)

Mục đích: Hiển thị dữ liệu cho người dùng.

Bao gồm:

- JSP files
- HTML
- CSS, JavaScript

Ví dụ:

```
<h1>Hello <%= user.getUsername() %>!/h1>
```

3. Controller (Điều khiển - Xử lý yêu cầu)

Mục đích: Tiếp nhận request, gọi Model xử lý, truyền kết quả cho View.

Bao gồm:

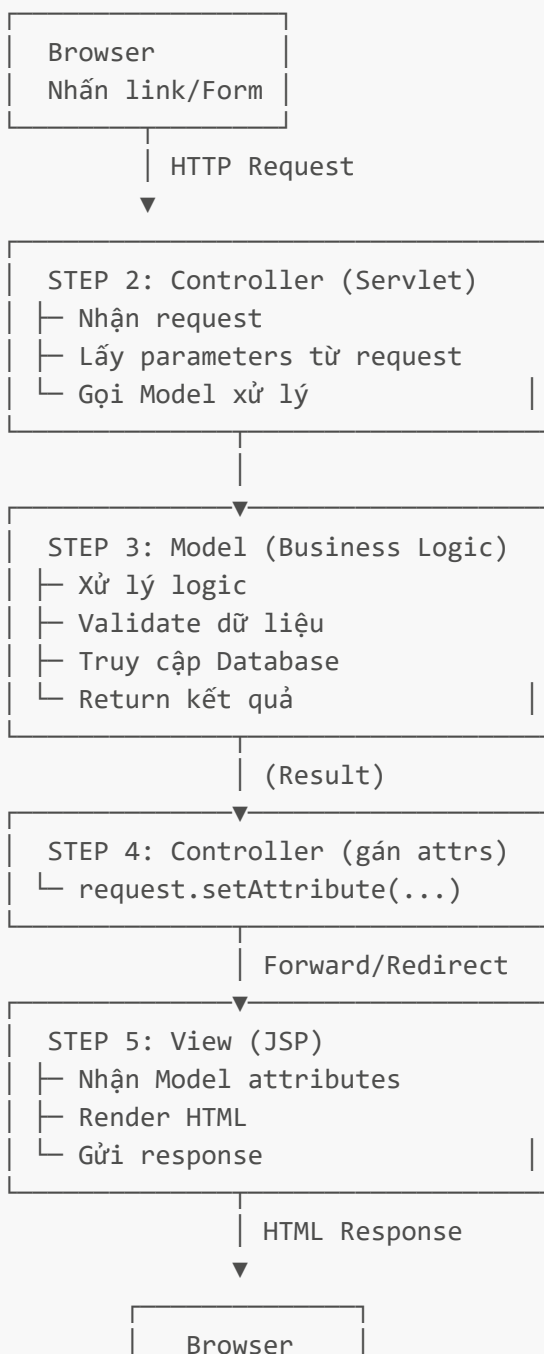
- Servlet
- Logic điều khiển
- Request handling

Ví dụ:

```
@WebServlet("/user/profile")
public class UserController extends HttpServlet {
    protected void doGet(...) {
        User user = UserDAO.findById(id);
        request.setAttribute("user", user);
        request.getRequestDispatcher("profile.jsp").forward(request, response);
    }
}
```

15.3 Quy trình MVC

STEP 1: User tương tác



Hiển thị trang

15.4 Lợi ích của MVC

Lợi ích	Giải thích
Tách biệt	Model, View, Controller độc lập
Bảo trì	Dễ debug, sửa code
Tái sử dụng	Model có thể dùng cho nhiều View
Scalability	Dễ mở rộng chức năng
Testing	Dễ viết unit test
Team work	Mỗi người làm phần riêng

15.5 Ví dụ So sánh: Với vs Không MVC

✗ Không MVC (Code lộn xộn)

```
<!-- Tất cả code trong JSP -->
<%@ page import="java.sql.*" %>

<%
    // Database connection
    Connection conn = DriverManager.getConnection("jdbc:mysql://localhost/db",
"user", "pass");

    // SQL query
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM users WHERE id=1");
    rs.next();
    String username = rs.getString("username");

    // HTML + Validation + Logic + Display (tất cả ở đây!)
%>

<h1>Hello <%= username %></h1>
```

Vấn đề:

- Code khó đọc
- Khó bảo trì
- Khó test
- Lộn xộn logic và hiển thị

☑ Với MVC (Code sạch)

Model - UserDao.java

```
public class UserDao {
    public User findById(int id) {
        // Database logic
    }
}

public class User {
    private String username;
    public String getUsername() { return username; }
}
```

Controller - UserServicelet.java

```
@WebServlet("/user")
public class UserServicelet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) {
        User user = UserDao.findById(1);
        req.setAttribute("user", user);
        req.getRequestDispatcher("user.jsp").forward(req, res);
    }
}
```

View - user.jsp

```
<h1>Hello ${ user.username }!</h1>
```

Lợi ích:

- ☑ Code sạch, dễ đọc
- ☑ Logic tách biệt
- ☑ Dễ bảo trì & test
- ☑ Tổ chức tốt

📌 Tóm tắt Session 15

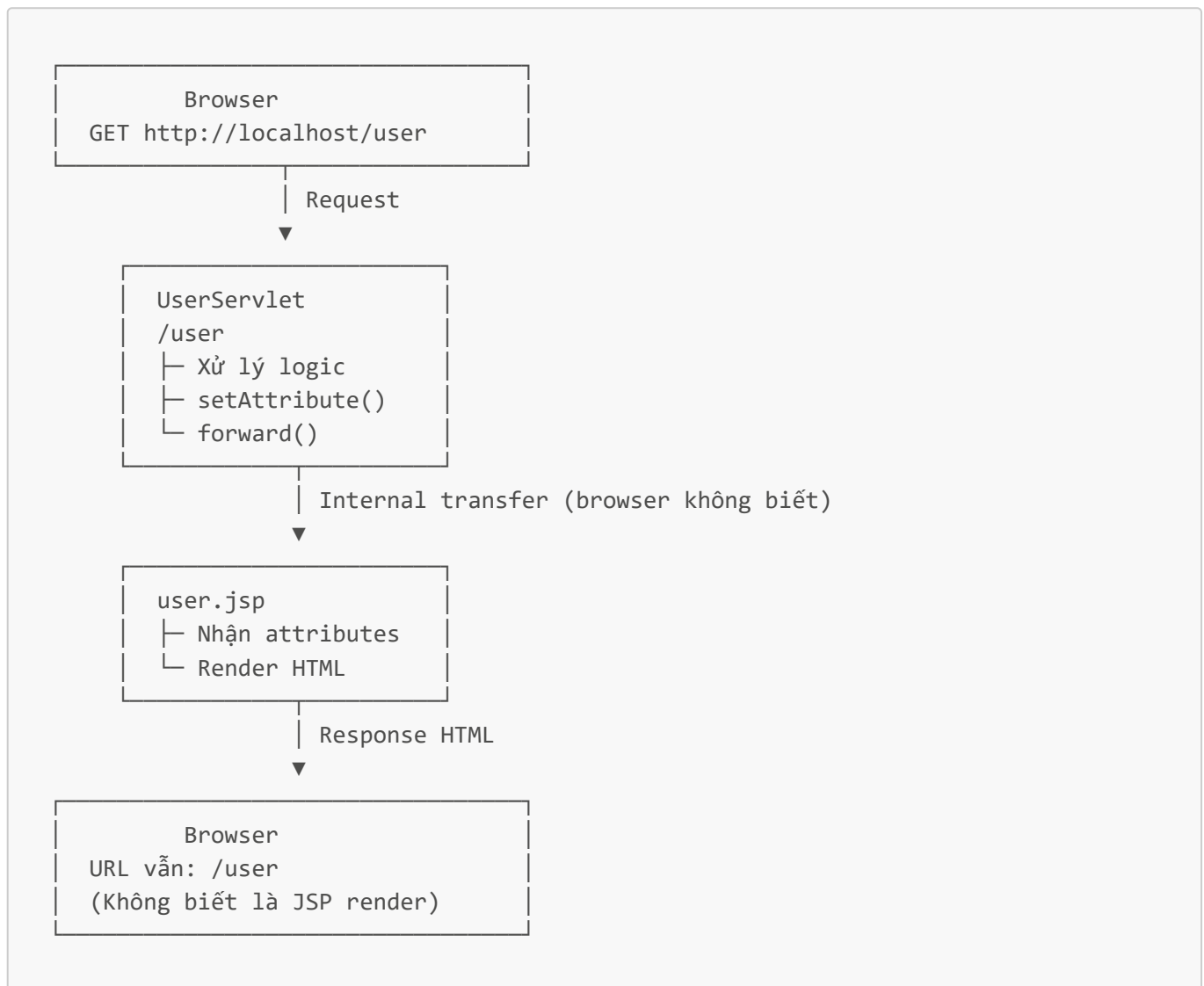
Thành phần	Trách nhiệm	Ví dụ
Model	Data + Logic	JavaBean, DAO, Business logic
View	Display	JSP, HTML, CSS
Controller	Request handling	Servlet, Processing

SESSION 16: Forward vs Redirect

16.1 Forward - RequestDispatcher

Định nghĩa

Forward = Chuyển request từ servlet này sang servlet/JSP khác **ở trên cùng server**, browser không biết.



Cú pháp

```
// Method 1: RequestDispatcher
RequestDispatcher dispatcher = request.getRequestDispatcher("path/to/file.jsp");
dispatcher.forward(request, response);

// Method 2: Rút gọn
request.getRequestDispatcher("path/to/file.jsp").forward(request, response);
```

Ví dụ

```
@WebServlet("/user/profile")
public class UserProfileServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        // 1. Lấy user ID từ parameter
        int userId = Integer.parseInt(request.getParameter("id"));

        // 2. Lấy data từ Model
        User user = UserDAO.findById(userId);

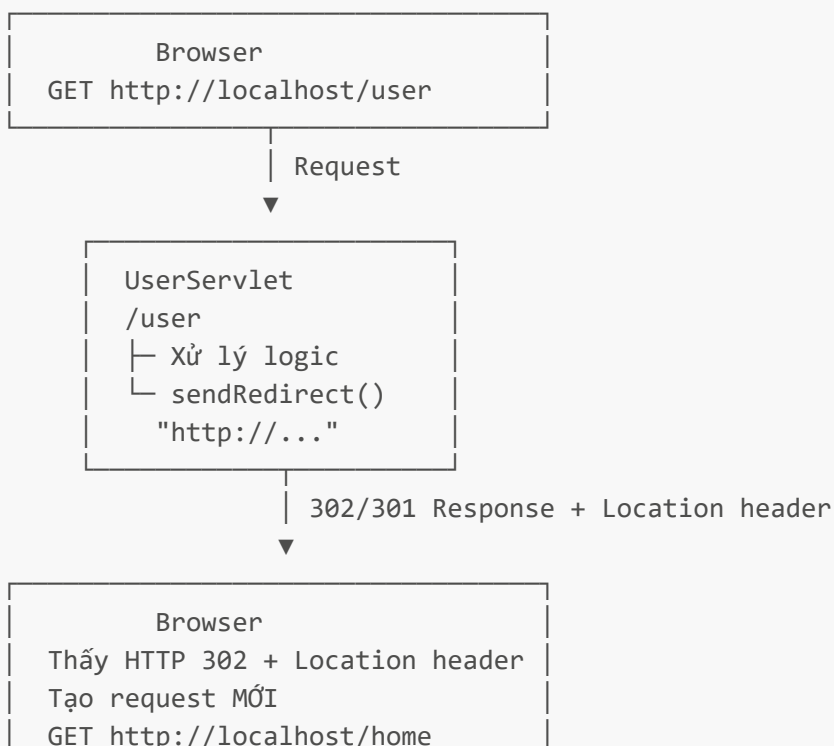
        // 3. Set attribute
        request.setAttribute("user", user);
        request.setAttribute("message", "User profile loaded");

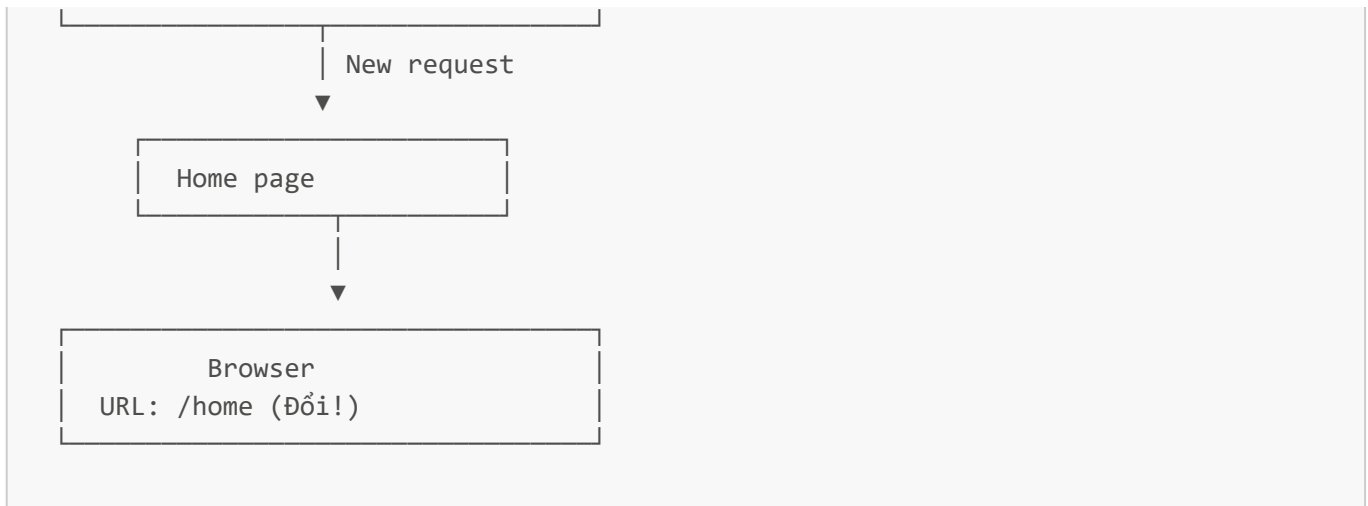
        // 4. Forward đến JSP
        request.getRequestDispatcher("user_profile.jsp").forward(request,
response);
    }
}
```

16.2 Redirect - response.sendRedirect()

Định nghĩa

Redirect = Server báo browser chuyển sang URL khác, browser thực hiện request mới.





Cú pháp

```
response.sendRedirect("path/to/page");  
response.sendRedirect("https://example.com");  
response.sendRedirect("/app/home");
```

Ví dụ

```
@WebServlet("/login")  
public class LoginServlet extends HttpServlet {  
  
    protected void doPost(HttpServletRequest request,  
                           HttpServletResponse response)  
        throws ServletException, IOException {  
  
        String username = request.getParameter("username");  
        String password = request.getParameter("password");  
  
        // Kiểm tra login  
        if (isLoginValid(username, password)) {  
            // 1. Lưu session  
            HttpSession session = request.getSession();  
            session.setAttribute("username", username);  
  
            // 2. Redirect đến home page  
            response.sendRedirect("/app/home");  
            // Browser sẽ thấy HTTP 302 và tạo request mới đến /app/home  
        } else {  
            // Redirect về login có error message  
            response.sendRedirect("/app/login?error=1");  
        }  
    }  
}
```

16.3 So sánh Forward vs Redirect

Yếu tố	Forward	Redirect
Nơi xử lý	Server	Browser
URL Browser	Không đổi	Đổi
HTTP Status	200 (forward nội bộ)	302/301
Attributes	Giữ lại (cùng request)	Mất (request mới)
Performance	Nhanh hơn	Chậm hơn (2 request)
Dùng khi	Chuyển sang JSP view	Chuyển sang trang khác
Ví dụ	Form → Servlet → JSP	After login → Home

16.4 Khi nào dùng Forward, khi nào dùng Redirect?

☒ Dùng Forward

```
// 1. Controller → View (hiển thị kết quả)
request.getRequestDispatcher("result.jsp").forward(request, response);

// 2. Sử dụng lại attributes
request.setAttribute("data", data);
request.getRequestDispatcher("display.jsp").forward(request, response);

// 3. Xử lý error
request.setAttribute("error", "Dữ liệu sai");
request.getRequestDispatcher("form.jsp").forward(request, response);
```

☒ Dùng Redirect

```
// 1. Sau thao tác thay đổi dữ liệu (Post-Redirect-Get pattern)
// Lưu dữ liệu
UserDAO.save(user);
// Redirect để tránh resubmit
response.sendRedirect("/app/users");

// 2. Đăng nhập thành công → Home
response.sendRedirect("/app/home");

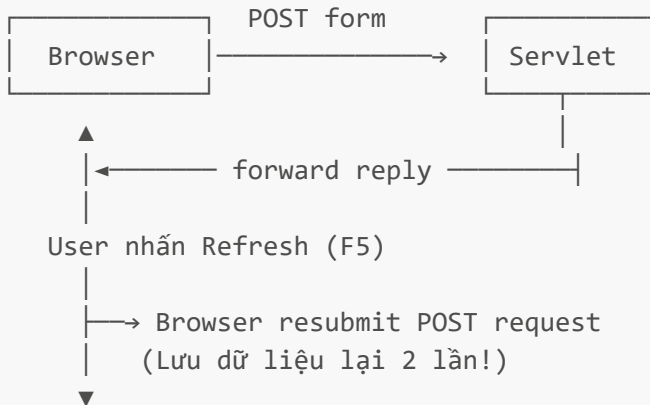
// 3. Chuyển hướng sang trang ngoài
response.sendRedirect("https://example.com");

// 4. Không có quyền → Redirect về login
response.sendRedirect("/app/login");
```

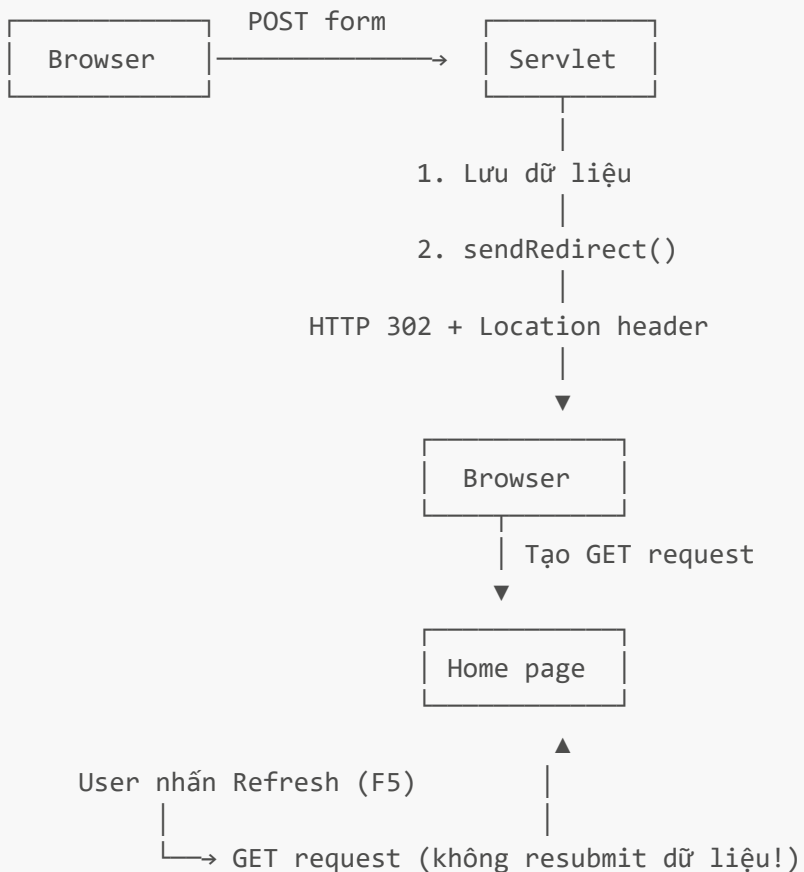
16.5 Ví dụ Thực Tế: Form Submission

Post-Redirect-Get Pattern (Tháo vòng lặp)

✗ Vấn đề:



☑ Giải pháp: POST-REDIRECT-GET



Code

```
@WebServlet("/product/add")
public class AddProductServlet extends HttpServlet {
```

```

protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws ServletException, IOException {

    String name = request.getParameter("name");
    String price = request.getParameter("price");

    try {
        // Validate
        if (name == null || price == null) {
            request.setAttribute("error", "Missing data");
            request.getRequestDispatcher("add_product.jsp")
                .forward(request, response);
            return;
        }

        // Lưu dữ liệu
        Product product = new Product(name, Double.parseDouble(price));
        ProductDAO.insert(product);

        // ☒ Redirect (tránh resubmit)
        response.sendRedirect("/app/products");

    } catch (Exception e) {
        request.setAttribute("error", e.getMessage());
        request.getRequestDispatcher("add_product.jsp")
            .forward(request, response);
    }
}

```

Tóm tắt Session 16

Yếu tố	Forward	Redirect
Forward	Internal transfer, URL không đổi	<code>request.getRequestDispatcher().forward()</code>
Redirect	Browser chuyển sang URL khác	<code>response.sendRedirect()</code>
Dùng	Controller → View	After action → List page

SESSION 17: JavaBeans

17.1 JavaBeans là gì?

JavaBeans = Một class Java chuẩn, theo một số quy tắc nhất định.

Đặc điểm JavaBeans

```

public class User {
    // 1. Có constructor không tham số (default)
    public User() {}

    // 2. Attributes private
    private String name;
    private int age;

    // 3. Getter/Setter public
    public String getName() { return name; }
    public void setName(String n) { this.name = n; }

    public int getAge() { return age; }
    public void setAge(int a) { this.age = a; }
}

```

Quy tắc JavaBeans

☒ ĐÚNG <pre> public class User ↓ public User() ↓ private int age ↓ public int getAge() public void setAge(int a) </pre>	☐ SAI <pre> public class User ↓ public User(String name) ↓ public int age ↓ Không có getter/setter </pre>
--	---

17.2 Getter/Setter Naming Convention

Quy tắc đặt tên:

Getter: get + FirstCapital(propertyName)
 Setter: set + FirstCapital(propertyName)

Ví dụ:

- Property: name → getName(), setName()
- Property: age → getAge(), setAge()
- Property: emailAddress → getEmailAddress(), setEmailAddress()
- Property: isActive → isActive(), setActive()

17.3 JavaBeans trong JSP

JSP có hỗ trợ đặc biệt cho JavaBeans qua `<jsp:useBean>`.

Tạo object JavaBean

```
<!-- Cách 1: Scriptlet --%>
<% User user = new User(); %>

<!-- Cách 2: JSP Tag (sạch hơn) --%>
<jsp:useBean id="user" class="com.example.User" scope="request" />
```

Set Property

```
<!-- Cách 1: Setter method --%>
<% user.setName("Nhân"); %>

<!-- Cách 2: JSP Tag --%>
<jsp:setProperty name="user" property="name" value="Nhân" />

<!-- Cách 3: Tự động từ parameter --%>
<jsp:setProperty name="user" property="name" param="username" />
```

Get Property

```
<!-- Cách 1: Getter method --%>
<p><%= user.getName() %></p>

<!-- Cách 2: EL --%>
<p>${ user.name }</p>

<!-- Cách 3: JSP Tag --%>
<jsp:getProperty name="user" property="name" />
```

17.4 Ví dụ Thực Tế: JavaBeans + JSP

Model - User.java (JavaBean)

```
package com.example.model;

public class User {
    private int id;
    private String name;
    private String email;
    private String phone;

    // Default constructor (bắt buộc)
    public User() {}
```

```
// Constructor with params (tùy chọn)
public User(int id, String name, String email, String phone) {
    this.id = id;
    this.name = name;
    this.email = email;
    this.phone = phone;
}

// Getters & Setters
public int getId() { return id; }
public void setId(int id) { this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }

public String getPhone() { return phone; }
public void setPhone(String phone) { this.phone = phone; }
}
```

Controller - UserServicelet.java

```
@WebServlet("/user")
public class UserServicelet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        // 1. Tạo User object (Model)
        User user = new User(1, "Nhân", "nhan@example.com", "0912345678");

        // 2. Set vào request scope
        request.setAttribute("user", user);

        // 3. Forward đến JSP
        request.getRequestDispatcher("user_profile.jsp")
            .forward(request, response);
    }
}
```

View - user_profile.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
```

```
<!DOCTYPE html>
<html>
<head><title>User Profile</title></head>
<body>
  <!-- Cách 1: Sử dụng getter --%>
  <h1><%= ((com.example.model.User)request.getAttribute("user")).getName() %>
</h1>

  <!-- Cách 2: Sử dụng EL (dễ hơn) --%>
  <div>
    <p>Name: ${ user.name }</p>
    <p>Email: ${ user.email }</p>
    <p>Phone: ${ user.phone }</p>
  </div>

  <!-- Cách 3: Sử dụng JSP useBean (cách cũ, ít dùng) --%>
  <jsp:useBean id="user2" class="com.example.model.User" scope="request" />
  <jsp:getProperty name="user2" property="name" />
</body>
</html>
```

17.5 Data Binding (Tự động gán dữ liệu)

JSP có thể tự động gán form parameters vào JavaBean properties.

```
<!-- Form HTML --%>
<form method="POST" action="register">
  <input type="text" name="name" />
  <input type="email" name="email" />
  <input type="tel" name="phone" />
  <button type="submit">Register</button>
</form>

<!-- Servlet --%>
<%
  User user = new User();

  // Phải gán thủ công
  user.setName(request.getParameter("name"));
  user.setEmail(request.getParameter("email"));
  user.setPhone(request.getParameter("phone"));
%>

<!-- JSP (Tự động binding) --%>
<jsp:useBean id="user" class="com.example.model.User" scope="request" />
<jsp:setProperty name="user" property="*" />
<!-- property="*" nghĩa là gán TẤT CẢ parameters vào properties cùng tên --%>
```

Tóm tắt Session 17

Yếu tố	Chi tiết
JavaBean	Class Java chuẩn với getter/setter
Default Constructor	<code>public ClassName() {}</code>
Naming	Property "name" → <code>getName()</code> , <code>setName()</code>
JSP useBean	<code><jsp:useBean ... /></code>
Data Binding	Tự động gán form data vào properties

SESSION 18: Controller Organization

18.1 Là sao phải tổ chức Controller?

Vấn đề: Nếu đặt tất cả Servlet ở 1 package, code sẽ lộn xộn.

☒ Cấu trúc xấu:

```
src/  
└─ com/example/  
    ├── UserServicelet.java  
    ├── ProductServlet.java  
    ├── OrderServlet.java  
    ├── UserDAO.java  
    ├── ProductDAO.java  
    ├── User.java  
    ├── Product.java  
    └─ ... (tất cả ở 1 package!)
```

☒ Cấu trúc tốt:

```
src/  
└─ com/example/  
    ├── servlet/  
    │   ├── UserServicelet.java  
    │   ├── ProductServlet.java  
    │   └─ OrderServlet.java  
    ├── model/  
    │   ├── User.java  
    │   ├── Product.java  
    │   └─ Order.java  
    ├── dao/  
    │   ├── UserDAO.java  
    │   ├── ProductDAO.java  
    │   └─ OrderDAO.java  
    └─ service/
```

```
├─ UserService.java
└─ ProductService.java
```

18.2 Cấu trúc Project MVC Hoàn Chỉnh

```
MyProjectApp/
├─ src/main/java/
│   └─ com/example/
│       ├── servlet/                (Controllers)
│       │   ├── HomeServlet.java
│       │   ├── UserServlet.java
│       │   ├── ProductServlet.java
│       │   └── OrderServlet.java
│       ├── controller/            (Tùy chọn: Base controller)
│       │   └── BaseController.java
│       ├── model/                 (Models)
│       │   ├── User.java
│       │   ├── Product.java
│       │   └── Order.java
│       ├── dao/                   (Data Access Objects)
│       │   ├── UserDAO.java
│       │   ├── ProductDAO.java
│       │   └── OrderDAO.java
│       ├── service/               (Business Logic)
│       │   ├── UserService.java
│       │   ├── ProductService.java
│       │   └── OrderService.java
│       ├── util/                  (Utilities)
│       │   ├── DatabaseConnection.java
│       │   ├── StringUtil.java
│       │   └── ValidationUtil.java
│       └─ filter/                 (Filters)
│           ├── AuthFilter.java
│           └── EncodingFilter.java
├─ src/main/webapp/
│   ├── WEB-INF/
│   │   └─ web.xml
│   ├── views/                    (JSP - Views)
│   │   ├── home.jsp
│   │   ├── user/
│   │   │   ├── list.jsp
│   │   │   ├── form.jsp
│   │   │   └── detail.jsp
```

```
| | | product/
| | | | list.jsp
| | | | detail.jsp
| | | | error.jsp
| | | css/
| | | | style.css
| | | | bootstrap.css
| | | js/
| | | | main.js
| | | | validation.js
| | | images/
| | | | logo.png
| lib/ (hoặc Maven: pom.xml)
| | mysql-connector-java.jar
| | jstl.jar
```

18.3 Base Controller (Tùy chọn)

Tạo class cha cho tất cả Controller để tránh code lặp.

```
package com.example.controller;

import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

public class BaseController extends HttpServlet {

    /**
     * Forward đến JSP
     */
    protected void forward(String path, HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {
        request.getRequestDispatcher(path).forward(request, response);
    }

    /**
     * Redirect
     */
    protected void redirect(String url, HttpServletResponse response)
        throws IOException {
        response.sendRedirect(url);
    }
}
```

```
/**
 * Set multiple attributes
 */
protected void setAttributes(HttpServletRequest request,
                             Map<String, Object> attrs) {
    for (String key : attrs.keySet()) {
        request.setAttribute(key, attrs.get(key));
    }
}

/**
 * Get int parameter
 */
protected int getIntParam(HttpServletRequest request, String paramName, int
defaultValue) {
    try {
        String value = request.getParameter(paramName);
        return value != null ? Integer.parseInt(value) : defaultValue;
    } catch (NumberFormatException e) {
        return defaultValue;
    }
}

/**
 * Get String parameter
 */
protected String getStringParam(HttpServletRequest request, String paramName)
{
    String value = request.getParameter(paramName);
    return value != null ? value.trim() : "";
}
}
```

18.4 Ví dụ: Servlet kế thừa BaseController

```
package com.example.servlet;

import com.example.controller.BaseController;
import com.example.model.User;
import com.example.service.UserService;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@WebServlet("/user")
```

```
public class UserServlet extends BaseController {

    private UserService userService = new UserService();

    @Override
    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        String action = getStringParam(request, "action");

        if ("list".equals(action)) {
            getList(request, response);
        } else if ("detail".equals(action)) {
            getDetail(request, response);
        } else {
            redirect("/app/user?action=list", response);
        }
    }

    private void getList(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        List<User> users = userService.getAllUsers();

        Map<String, Object> attrs = new HashMap<>();
        attrs.put("users", users);
        attrs.put("title", "User List");

        setAttributes(request, attrs);
        forward("/views/user/list.jsp", request, response);
    }

    private void getDetail(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        int userId = getIntParam(request, "id", -1);

        if (userId <= 0) {
            redirect("/app/user?action=list", response);
            return;
        }

        User user = userService.getUserById(userId);

        if (user == null) {
            request.setAttribute("error", "User not found");
            forward("/views/error.jsp", request, response);
            return;
        }

        request.setAttribute("user", user);
    }
}
```

```
        forward("/views/user/detail.jsp", request, response);  
    }  
}
```

18.5 URL Mapping Pattern

Thường dùng **Single Controller + Action Parameter**:

☒ Cách cũ (các Servlet khác nhau):
/UserServlet
/ProductServlet
/OrderServlet

☒ Cách mới (1 Servlet, action parameter):
/user?action=list
/user?action=detail&id=1
/user?action=create
/user?action=edit&id=1

/product?action=list
/product?action=detail&id=1

/order?action=list
/order?action=detail&id=1

Tóm tắt Session 18

Cấu trúc Controller:

- ☒ Tách servlet, model, dao, service vào package riêng
- ☒ Sử dụng BaseController để tránh code lặp
- ☒ Dùng action parameter thay vì nhiều Servlet
- ☒ Organized code → Dễ bảo trì

☒ SESSION 19: Validation (Kiểm thử Dữ liệu)

19.1 Tại sao cần Validation?

Vấn đề:

- User có thể nhập sai dữ liệu
- Dữ liệu không hợp lệ → Database error
- Security issue (SQL injection, XSS, v.v.)

Giải pháp: Validate dữ liệu trước khi lưu.

19.2 Client-side vs Server-side Validation

Client-side Validation (JavaScript)

```
<form onsubmit="return validateForm()">
  <input type="text" id="name" required minlength="3" maxlength="50" />
  <input type="email" id="email" required />
  <input type="tel" id="phone" required />
  <button type="submit">Submit</button>
</form>

<script>
function validateForm() {
  let name = document.getElementById("name").value;
  if (name.length < 3) {
    alert("Name too short!");
    return false;
  }
  return true;
}
</script>
```

Lợi ích: Nhanh, không cần gửi request **Vấn đề:** Browser có thể vô hiệu hóa JavaScript

Server-side Validation (Java)

```
public class ValidationUtil {

    public static String validateEmail(String email) {
        if (email == null || email.isEmpty()) {
            return "Email is required";
        }
        if (!email.matches("^[A-Za-z0-9+_.-]+@(.+)$")) {
            return "Invalid email format";
        }
        return null; // Valid
    }

    public static String validatePhone(String phone) {
        if (phone == null || phone.isEmpty()) {
            return "Phone is required";
        }
        if (!phone.matches("^[0-9]{10,11}$")) {
            return "Phone must be 10-11 digits";
        }
        return null; // Valid
    }
}
```

Lợi ích: An toàn, bắt buộc phải thực hiện **Vấn đề:** Chậm hơn (request → server → response)

☑ Best practice: Cả hai!

```
Browser (Client-side check)
  ↓
Nhanh nhưng có thể vô hiệu hóa
  ↓
└─> Server (Server-side check)
      ↓
      An toàn, bắt buộc
      ↓
      Lưu database
```

19.3 Server-side Validation Examples

Validate Name

```
public static String validateName(String name) {
    if (name == null || name.trim().isEmpty()) {
        return "Name is required";
    }

    name = name.trim();

    if (name.length() < 3) {
        return "Name must be at least 3 characters";
    }

    if (name.length() > 50) {
        return "Name must not exceed 50 characters";
    }

    if (!name.matches("[a-zA-Z\\s]+$")) {
        return "Name can only contain letters and spaces";
    }

    return null; // Valid
}
```

Validate Email

```
public static String validateEmail(String email) {
    if (email == null || email.trim().isEmpty()) {
        return "Email is required";
    }
}
```

```
email = email.trim();

if (!email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Z|a-z]{2,}$")) {
    return "Invalid email format";
}

return null; // Valid
}
```

Validate Password

```
public static String validatePassword(String password) {
    if (password == null || password.isEmpty()) {
        return "Password is required";
    }

    if (password.length() < 6) {
        return "Password must be at least 6 characters";
    }

    if (password.length() > 20) {
        return "Password must not exceed 20 characters";
    }

    if (!password.matches(".*[0-9].*")) {
        return "Password must contain at least one digit";
    }

    if (!password.matches(".*[a-z].*")) {
        return "Password must contain at least one lowercase letter";
    }

    return null; // Valid
}
```

Validate Integer

```
public static String validateAge(String ageStr) {
    if (ageStr == null || ageStr.trim().isEmpty()) {
        return "Age is required";
    }

    try {
        int age = Integer.parseInt(ageStr.trim());

        if (age < 0 || age > 120) {
            return "Age must be between 0 and 120";
        }
    }
}
```

```
        return null; // Valid
    } catch (NumberFormatException e) {
        return "Age must be a valid number";
    }
}
```

19.4 Validation trong Servlet

```
@WebServlet("/register")
public class RegisterServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        // 1. Lấy dữ liệu
        String fullName = request.getParameter("fullName");
        String email = request.getParameter("email");
        String phone = request.getParameter("phone");
        String password = request.getParameter("password");
        String confirmPassword = request.getParameter("confirmPassword");

        // 2. Validate từng field
        String errors = "";

        String nameError = ValidationUtil.validateName(fullName);
        if (nameError != null) {
            errors += nameError + "\\n";
        }

        String emailError = ValidationUtil.validateEmail(email);
        if (emailError != null) {
            errors += emailError + "\\n";
        }

        String phoneError = ValidationUtil.validatePhone(phone);
        if (phoneError != null) {
            errors += phoneError + "\\n";
        }

        String passwordError = ValidationUtil.validatePassword(password);
        if (passwordError != null) {
            errors += passwordError + "\\n";
        }

        // Validate confirm password
        if (!password.equals(confirmPassword)) {
            errors += "Passwords do not match\\n";
        }
    }
}
```

```
// 3. Nếu có lỗi → quay lại form
if (!errors.isEmpty()) {
    request.setAttribute("error", errors);
    request.setAttribute("fullName", fullName);
    request.setAttribute("email", email);
    request.setAttribute("phone", phone);
    request.getRequestDispatcher("register.jsp")
        .forward(request, response);
    return;
}

// 4. Dữ liệu hợp lệ → lưu
try {
    User user = new User(fullName, email, phone, password);
    UserDAO.insert(user);

    response.sendRedirect("/app/register?success=1");
} catch (Exception e) {
    request.setAttribute("error", "Database error: " + e.getMessage());
    request.getRequestDispatcher("register.jsp")
        .forward(request, response);
}
}
```

19.5 Validation Utility Class

```
package com.example.util;

import java.util.HashMap;
import java.util.Map;

public class ValidationUtil {

    /**
     * Validate user registration data
     */
    public static Map<String, String> validateRegistration(
        String fullName, String email, String phone,
        String password, String confirmPassword) {

        Map<String, String> errors = new HashMap<>();

        // Name
        String nameError = validateName(fullName);
        if (nameError != null) errors.put("fullName", nameError);

        // Email
        String emailError = validateEmail(email);
```

```
        if (emailError != null) errors.put("email", emailError);

        // Phone
        String phoneError = validatePhone(phone);
        if (phoneError != null) errors.put("phone", phoneError);

        // Password
        String passwordError = validatePassword(password);
        if (passwordError != null) errors.put("password", passwordError);

        // Confirm password
        if (password == null || !password.equals(confirmPassword)) {
            errors.put("confirmPassword", "Passwords do not match");
        }

        return errors;
    }

    /**
     * Validate name
     */
    public static String validateName(String name) {
        if (name == null || name.trim().isEmpty()) {
            return "Name is required";
        }
        name = name.trim();
        if (name.length() < 3) return "Name too short (min 3)";
        if (name.length() > 50) return "Name too long (max 50)";
        return null;
    }

    // ... (các validation khác)
}
```

Tóm tắt Session 19

Validation:

- ☒ Client-side: Nhanh (nhưng có thể vô hiệu hóa)
- ☒ Server-side: An toàn (BẮT BUỘC)
- ☒ Validate từng field
- ☒ Trả lỗi về form, giữ dữ liệu đã nhập

SESSION 20: Error Handling

20.1 Loại Error

Browser Request



Web Application	← Lỗi ở đây?
1. IOException (network) 2. NullPointerException 3. SQLException (database) 4. 404 - Not Found 5. 500 - Server Error	



Response Error Page

20.2 Custom Error Page trong web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0" xmlns="...">

    <display-name>My App</display-name>

    <!-- Error handling -->

    <!-- Lỗi 404 - Not Found -->
    <error-page>
        <error-code>404</error-code>
        <location>/error/404.jsp</location>
    </error-page>

    <!-- Lỗi 500 - Server Error -->
    <error-page>
        <error-code>500</error-code>
        <location>/error/500.jsp</location>
    </error-page>

    <!-- Exception - NullPointerException -->
    <error-page>
        <exception-type>java.lang.NullPointerException</exception-type>
        <location>/error/null_pointer.jsp</location>
    </error-page>

    <!-- Exception - SQLException -->
    <error-page>
        <exception-type>java.sql.SQLException</exception-type>
        <location>/error/database.jsp</location>
    </error-page>

    <!-- Exception chung -->
    <error-page>
```

```
<exception-type>java.lang.Exception</exception-type>
<location>/error/generic.jsp</location>
</error-page>

</web-app>
```

20.3 Custom Error Pages

error/404.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>

<!DOCTYPE html>
<html>
<head>
    <title>404 - Not Found</title>
    <style>
        body {
            font-family: Arial;
            text-align: center;
            padding: 50px;
            background-color: #f5f5f5;
        }
        .error-container {
            background: white;
            padding: 40px;
            border-radius: 8px;
            max-width: 600px;
            margin: 0 auto;
        }
        .error-code {
            font-size: 72px;
            color: #dc3545;
            font-weight: bold;
            margin: 0;
        }
        .error-message {
            font-size: 24px;
            color: #333;
            margin: 20px 0;
        }
        .error-description {
            color: #666;
            margin: 20px 0;
        }
        a {
            display: inline-block;
            margin-top: 20px;
            padding: 10px 20px;
```

```

        background-color: #007bff;
        color: white;
        text-decoration: none;
        border-radius: 5px;
    }
</style>
</head>
<body>
    <div class="error-container">
        <p class="error-code">404</p>
        <p class="error-message">Page Not Found</p>
        <p class="error-description">
            The page you're looking for doesn't exist or has been moved.
        </p>
        <a href="/">< Back to Home</a>
    </div>
</body>
</html>

```

error/500.jsp

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"
    isErrorPage="true" %>

<!DOCTYPE html>
<html>
<head>
    <title>500 - Server Error</title>
    <style>
        /* Same styling as 404 */
    </style>
</head>
<body>
    <div class="error-container">
        <p class="error-code">500</p>
        <p class="error-message">Server Error</p>
        <p class="error-description">
            Something went wrong on our server. Please try again later.
        </p>

        <!-- Chỉ hiển thị error detail ở development mode -->
        <% if (exception != null) { %>
            <h3 style="text-align: left; color: #dc3545;">Error Details:</h3>
            <pre style="text-align: left; background: #f5f5f5; padding: 10px;
                border-radius: 5px; overflow: auto;">
                <%= exception.getMessage() %>
            </pre>
        <% } %>

        <a href="/">< Back to Home</a>
    </div>
</body>
</html>

```

```
</div>
</body>
</html>
```

20.4 Try-Catch trong Servlet

```
@WebServlet("/user")
public class UserServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        try {
            int userId = Integer.parseInt(request.getParameter("id"));
            User user = UserDAO.findById(userId);

            if (user == null) {
                // 404
                response.sendError(HttpServletResponse.SC_NOT_FOUND,
                                "User not found");
                return;
            }

            request.setAttribute("user", user);
            request.getRequestDispatcher("user.jsp").forward(request, response);

        } catch (NumberFormatException e) {
            request.setAttribute("error", "Invalid user ID");
            request.getRequestDispatcher("error/400.jsp").forward(request,
response);

        } catch (SQLException e) {
            // 500 Error
            request.setAttribute("error", "Database error: " + e.getMessage());
            request.getRequestDispatcher("error/500.jsp").forward(request,
response);

        } catch (Exception e) {
            // Generic error
            request.setAttribute("error", "Unexpected error: " + e.getMessage());
            request.getRequestDispatcher("error/500.jsp").forward(request,
response);
        }
    }
}
```

20.5 Logger cho Error

```
import java.util.logging.Logger;
import java.util.logging.Level;

public class UserServlet extends HttpServlet {

    private static final Logger logger =
        Logger.getLogger(UserServlet.class.getName());

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        try {
            int userId = Integer.parseInt(request.getParameter("id"));
            User user = UserDAO.findById(userId);

            // Log success
            logger.info("User " + userId + " retrieved successfully");

        } catch (NumberFormatException e) {
            logger.warning("Invalid user ID format: " +
                request.getParameter("id"));
        } catch (SQLException e) {
            logger.log(Level.SEVERE, "Database error", e);
        } catch (Exception e) {
            logger.log(Level.SEVERE, "Unexpected error", e);
        }
    }
}
```

Tóm tắt Session 20

Error Handling:

- ☒ Custom error page trong web.xml
- ☒ Try-catch trong Servlet
- ☒ Gửi error code (404, 500)
- ☒ Log errors cho debugging

SESSION 21: Mini Project - CRUD Flow Skeleton

21.1 Yêu cầu Dự Án

Tạo **Student Management System** với:

- Create (Tạo sinh viên)
- Read (Xem danh sách, chi tiết)
- Update (Sửa thông tin)
- Delete (Xóa sinh viên)

Lưu ý: Chỉ lưu vào file, không dùng database.

21.2 Cấu trúc Project

```
StudentManagementApp/
├── src/main/java/com/example/
│   ├── servlet/
│   │   ├── StudentServlet.java
│   │   └── HomeServlet.java
│   ├── model/
│   │   └── Student.java
│   ├── service/
│   │   └── StudentService.java
│   ├── util/
│   │   ├── ValidationUtil.java
│   │   └── FileUtil.java
│   └── controller/
│       └── BaseController.java
├── src/main/webapp/
│   ├── WEB-INF/
│   │   └── web.xml
│   ├── views/
│   │   ├── home.jsp
│   │   ├── student/
│   │   │   ├── list.jsp
│   │   │   ├── form.jsp
│   │   │   ├── detail.jsp
│   │   │   └── success.jsp
│   │   └── error/
│   │       ├── 404.jsp
│   │       └── 500.jsp
│   ├── css/
│   │   └── style.css
│   └── js/
│       └── validation.js
└── build/StudentManagementApp.war
```

21.3 Model - Student.java

```
package com.example.model;

import java.io.Serializable;

public class Student implements Serializable {
    private static final long serialVersionUID = 1L;

    private int id;
    private String studentId;
    private String fullName;
    private String email;
    private String phone;
    private String major;
    private int yearOfBirth;

    // Constructors
    public Student() {}

    public Student(int id, String studentId, String fullName, String email,
        String phone, String major, int yearOfBirth) {
        this.id = id;
        this.studentId = studentId;
        this.fullName = fullName;
        this.email = email;
        this.phone = phone;
        this.major = major;
        this.yearOfBirth = yearOfBirth;
    }

    // Getters & Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getStudentId() { return studentId; }
    public void setStudentId(String studentId) { this.studentId = studentId; }

    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getPhone() { return phone; }
    public void setPhone(String phone) { this.phone = phone; }

    public String getMajor() { return major; }
    public void setMajor(String major) { this.major = major; }

    public int getYearOfBirth() { return yearOfBirth; }
    public void setYearOfBirth(int yearOfBirth) { this.yearOfBirth = yearOfBirth; }
}

// Helper method
```

```
public int getAge() {
    return 2025 - yearOfBirth;
}

@Override
public String toString() {
    return id + "|" + studentId + "|" + fullName + "|" + email + "|" +
        phone + "|" + major + "|" + yearOfBirth;
}

// Parse từ string
public static Student fromString(String line) {
    String[] parts = line.split("\\|");
    if (parts.length == 7) {
        return new Student(
            Integer.parseInt(parts[0]), parts[1], parts[2], parts[3],
            parts[4], parts[5], Integer.parseInt(parts[6])
        );
    }
    return null;
}
}
```

21.4 Service - StudentService.java

```
package com.example.service;

import com.example.model.Student;
import java.util.*;

public class StudentService {

    // Lưu trong memory (tạm thời)
    private static List<Student> students = new ArrayList<>();
    private static int nextId = 1;

    // Initialize sample data
    static {
        students.add(new Student(nextId++, "23001", "Nguyễn Văn A",
            "a@example.com", "0912345678", "Computer
Science", 2005));
        students.add(new Student(nextId++, "23002", "Trần Thị B",
            "b@example.com", "0923456789", "Business",
2006));
    }

    /**
     * Lấy tất cả sinh viên
     */
    public List<Student> getAllStudents() {
```

```
        return new ArrayList<>(students);
    }

    /**
     * Lấy sinh viên theo ID
     */
    public Student getStudentById(int id) {
        return students.stream()
            .filter(s -> s.getId() == id)
            .findFirst()
            .orElse(null);
    }

    /**
     * Thêm sinh viên
     */
    public void addStudent(Student student) {
        student.setId(nextId++);
        students.add(student);
    }

    /**
     * Cập nhật sinh viên
     */
    public boolean updateStudent(Student student) {
        Student existing = getStudentById(student.getId());
        if (existing != null) {
            students.remove(existing);
            students.add(student);
            return true;
        }
        return false;
    }

    /**
     * Xóa sinh viên
     */
    public boolean deleteStudent(int id) {
        return students.removeIf(s -> s.getId() == id);
    }

    /**
     * Kiểm tra student ID đã tồn tại chưa
     */
    public boolean isStudentIdExists(String studentId) {
        return students.stream()
            .anyMatch(s -> s.getStudentId().equals(studentId));
    }
}
```

21.5 Controller - StudentServlet.java

```
package com.example.servlet;

import com.example.controller.BaseController;
import com.example.model.Student;
import com.example.service.StudentService;
import com.example.util.ValidationUtil;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@WebServlet("/student")
public class StudentServlet extends BaseController {

    private StudentService studentService = new StudentService();

    @Override
    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        String action = getStringParam(request, "action");

        if ("list".equals(action)) {
            getList(request, response);
        } else if ("detail".equals(action)) {
            getDetail(request, response);
        } else if ("form".equals(action)) {
            getForm(request, response);
        } else if ("delete".equals(action)) {
            deleteStudent(request, response);
        } else {
            redirect("/app/student?action=list", response);
        }
    }

    @Override
    protected void doPost(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        String action = getStringParam(request, "action");

        if ("create".equals(action)) {
            createStudent(request, response);
        } else if ("update".equals(action)) {
            updateStudent(request, response);
        }
    }
}
```

```
}

// GET: Danh sách sinh viên
private void getList(HttpServletRequest request,
                    HttpServletResponse response)
    throws ServletException, IOException {

    List<Student> students = studentService.getAllStudents();

    Map<String, Object> attrs = new HashMap<>();
    attrs.put("students", students);
    attrs.put("totalCount", students.size());

    setAttributes(request, attrs);
    forward("/views/student/list.jsp", request, response);
}

// GET: Chi tiết sinh viên
private void getDetail(HttpServletRequest request,
                     HttpServletResponse response)
    throws ServletException, IOException {

    int id = getIntParam(request, "id", -1);

    if (id <= 0) {
        request.setAttribute("error", "Invalid student ID");
        forward("/views/error/400.jsp", request, response);
        return;
    }

    Student student = studentService.getStudentById(id);

    if (student == null) {
        response.sendError(HttpServletResponse.SC_NOT_FOUND, "Student not
found");
        return;
    }

    request.setAttribute("student", student);
    forward("/views/student/detail.jsp", request, response);
}

// GET: Form tạo/sửa sinh viên
private void getForm(HttpServletRequest request,
                    HttpServletResponse response)
    throws ServletException, IOException {

    String mode = getStringParam(request, "mode"); // "create" hoặc "edit"

    if ("edit".equals(mode)) {
        int id = getIntParam(request, "id", -1);
        Student student = studentService.getStudentById(id);

        if (student == null) {
```

```
        response.sendError(HttpServletResponse.SC_NOT_FOUND);
        return;
    }

    request.setAttribute("student", student);
    request.setAttribute("mode", "edit");
} else {
    request.setAttribute("mode", "create");
}

forward("/views/student/form.jsp", request, response);
}

// POST: Tạo sinh viên mới
private void createStudent(HttpServletRequest request,
                          HttpServletResponse response)
    throws ServletException, IOException {

    String fullName = getStringParam(request, "fullName");
    String studentId = getStringParam(request, "studentId");
    String email = getStringParam(request, "email");
    String phone = getStringParam(request, "phone");
    String major = getStringParam(request, "major");
    String yearStr = getStringParam(request, "yearOfBirth");

    // Validate
    Map<String, String> errors = new HashMap<>();

    if (fullName.isEmpty()) {
        errors.put("fullName", "Full name is required");
    }

    if (studentId.isEmpty()) {
        errors.put("studentId", "Student ID is required");
    } else if (studentService.isStudentIdExists(studentId)) {
        errors.put("studentId", "Student ID already exists");
    }

    String emailError = ValidationUtil.validateEmail(email);
    if (emailError != null) {
        errors.put("email", emailError);
    }

    String phoneError = ValidationUtil.validatePhone(phone);
    if (phoneError != null) {
        errors.put("phone", phoneError);
    }

    if (major.isEmpty()) {
        errors.put("major", "Please select a major");
    }

    // Nếu có lỗi → quay lại form
    if (!errors.isEmpty()) {
```

```
        request.setAttribute("errors", errors);
        request.setAttribute("fullName", fullName);
        request.setAttribute("studentId", studentId);
        request.setAttribute("email", email);
        request.setAttribute("phone", phone);
        request.setAttribute("major", major);
        request.setAttribute("mode", "create");
        forward("/views/student/form.jsp", request, response);
        return;
    }

    // Tạo student
    try {
        int year = Integer.parseInt(yearStr);
        Student student = new Student(0, studentId, fullName, email, phone,
major, year);
        studentService.addStudent(student);

        request.setAttribute("message", "Student created successfully");
        forward("/views/student/success.jsp", request, response);

    } catch (Exception e) {
        request.setAttribute("error", "Error creating student: " +
e.getMessage());
        forward("/views/error/500.jsp", request, response);
    }
}

// POST: Cập nhật sinh viên
private void updateStudent(HttpServletRequest request,
                        HttpServletResponse response)
                        throws ServletException, IOException {

    // Tương tự createStudent
    // ...
}

// GET: Xóa sinh viên
private void deleteStudent(HttpServletRequest request,
                        HttpServletResponse response)
                        throws ServletException, IOException {

    int id = getIntParam(request, "id", -1);

    if (id <= 0) {
        response.sendError(HttpServletResponse.SC_BAD_REQUEST);
        return;
    }

    boolean deleted = studentService.deleteStudent(id);

    if (deleted) {
        response.sendRedirect("/app/student?action=list&message=deleted");
    } else {
```

```

        response.sendError(HttpServletResponse.SC_NOT_FOUND);
    }
}
}

```

21.6 Views - list.jsp

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<!DOCTYPE html>
<html>
<head>
    <title>Student Management</title>
    <style>
        body { font-family: Arial; margin: 0; padding: 20px; background: #f5f5f5; }
        .container { max-width: 1000px; margin: 0 auto; background: white; padding: 20px; border-radius: 8px; }
        h1 { color: #333; }
        .btn { padding: 8px 15px; margin: 5px; border: none; border-radius: 4px; cursor: pointer; }
        .btn-primary { background: #007bff; color: white; }
        .btn-danger { background: #dc3545; color: white; }
        table { width: 100%; border-collapse: collapse; margin-top: 20px; }
        th, td { padding: 10px; border: 1px solid #ddd; text-align: left; }
        th { background: #f0f0f0; font-weight: bold; }
        tr:hover { background: #f9f9f9; }
        .message { background: #d4edda; color: #155724; padding: 10px; border-radius: 4px; margin-bottom: 20px; }
    </style>
</head>
<body>
    <div class="container">
        <h1>📖 Student Management System</h1>

        <c:if test="${ not empty param.message }">
            <div class="message">
                ☒ Operation completed successfully!
            </div>
        </c:if>

        <a href="/app/student?action=form&mode=create" class="btn btn-primary">
            + New Student
        </a>
        <a href="/app/" class="btn">Home</a>

        <table>
            <thead>

```

```

        <tr>
            <th>ID</th>
            <th>Student ID</th>
            <th>Full Name</th>
            <th>Email</th>
            <th>Major</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        <c:choose>
            <c:when test="${ empty students }">
                <tr>
                    <td colspan="6" style="text-align: center; color:
#999;">
                        No students found
                    </td>
                </tr>
            </c:when>
            <c:otherwise>
                <c:forEach var="student" items="${ students }">
                    <tr>
                        <td>${ student.id }</td>
                        <td>${ student.studentId }</td>
                        <td>
                            <a href="/app/student?action=detail&id=${
student.id }"
                                style="color: #007bff; text-decoration:
none;">
                                ${ student.fullName }
                            </a>
                        </td>
                        <td>${ student.email }</td>
                        <td>${ student.major }</td>
                        <td>
                            <a href="/app/student?
action=form&mode=edit&id=${ student.id }"
                                class="btn" style="background: #ffc107;
padding: 5px 10px;">
                                Edit
                            </a>
                            <a href="/app/student?action=delete&id=${
student.id }"
                                class="btn btn-danger"
                                onclick="return confirm('Delete this
student?');"
                                style="padding: 5px 10px;">
                                Delete
                            </a>
                        </td>
                    </tr>
                </c:forEach>
            </c:otherwise>
        </c:choose>

```

```
        </tbody>
    </table>

    <p style="margin-top: 20px; color: #666;">
        Total: <strong>${ totalCount }</strong> students
    </p>
</div>
</body>
</html>
```

21.7 Deploy & Chạy

```
# Build & Deploy
jar cvf StudentManagementApp.war -C webapp .

# Copy to Tomcat
copy StudentManagementApp.war C:\apache-tomcat-10.1.x\webapps\

# Access
http://localhost:8080/StudentManagementApp/
```

Hoàn thành Tuần 3!

Bạn đã học:

- ☒ MVC Pattern (Model, View, Controller)
- ☒ Forward vs Redirect
- ☒ JavaBeans (POJO)
- ☒ Controller Organization (Cấu trúc project)
- ☒ Server-side Validation
- ☒ Error Handling (Custom error pages)
- ☒ Mini Project - CRUD System (In-memory)

Bước tiếp theo: Tuần 4 - Database Integration (JDBC + DAO)

Tài liệu Tham Khảo

- Oracle Servlet & JSP Documentation
- Design Patterns in Java
- Clean Code by Robert C. Martin
- MVC Architecture Best Practices

Tài liệu được biên soạn bởi: Senior Java Developer Ngày cập nhật: 2025 Phiên bản: 1.0